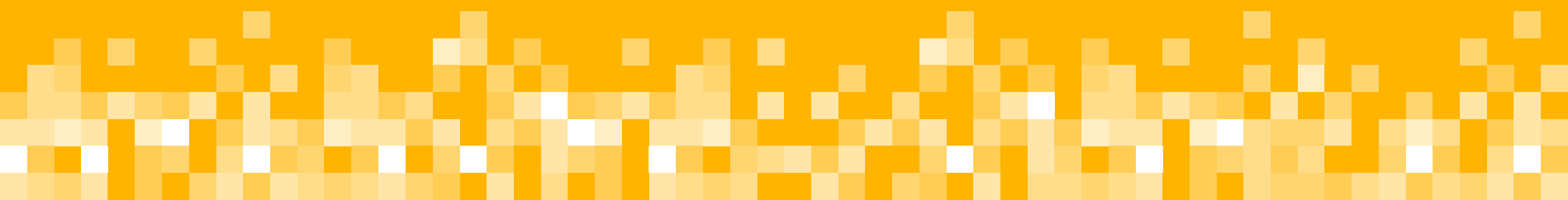


Moduł 3

Refaktoryzacja do wzorca Aggregate

Ćwiczenia praktyczne



Zadanie:

Oddzielenie decyzji od konsekwencji

Wykonaj komendę:

```
git checkout decyzje-zadanie
```

w odpowiednim projekcie:

- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-java>
- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-php>
- ➔ <https://github.com/legacyfighter/dodatkowe-zadania-csharp>

Na chwilę zmieniamy problem biznesowy na model zarządzania urlopami. Spójrz na klasę `legacyfighter.leave.LeaveService`. Obsługuje ona wnioski urlopowe za pomocą jedynej publicznej metody, `requestPaidDaysOff`. Logika udzielania urlopów wygląda następująco:

- 1) Jeśli jest się pracownikiem o wysokiej wydajności ("PERFORMER"), to można wykorzystać zawsze 26 dni (decyzja = Approved). Od 26 do 45 dni wnioski mogą być zaakceptowane, ale najpierw muszą być manualnie zweryfikowane przez managera ds. eskalacji (decyzja = Manual). Każdy wniosek, który przekracza 45 dni zostaje odrzucony (decyzja = Denied). Odrzucenie skutkuje wysłaniem stosownego e-maila.
- 2) Jeśli jest się regularnym pracownikiem ("REGULAR"), wnioski powyżej 26 dni są odrzucane, do 26 dni akceptowane. Odrzucenie skutkuje wysłaniem stosownego e-maila.

- 3) Jeśli jest się pracownikiem o niskiej wydajności ("SLACKER"), wszystkie wnioski są odrzucane, odrzucenie skutkuje wysłaniem stosownego e-maila.

Akceptacja wniosku skutkuje zapisaniem do bazy nowego stanu (liczby dni) oraz wysłaniem wiadomości na szynę danych.

1. Wypisz odpowiedzialności klasy wraz z ich lokalizacją w kodzie, biorąc pod uwagę:

- a) Najistotniejsza logika to ta, która sprawdza czy pracownik z odpowiednią wydajnością dostaje (lub nie) urlop. Chcemy to przetestować.
- b) Manager eskalacji (`EscalationManager.java`) to **płatna usługa**, chcemy ją wołać tylko wtedy, kiedy to jest konieczne. Chcemy to przetestować.
- c) Wysłany e-mail i jego treść to bardzo ważne aspekty zaproponowane przez nowego Agile Coacha - mają wpłynąć na morale osób, które nie mogą jechać na wakacje. Chcemy to przetestować.

2. Nazwij konkretne problemy, które może powodować ta klasa. Poniżej kilka problemów, z którymi boryka się zespół deweloperski:

- a) Bardzo częste zmiany związane z infrastrukturą (schemat bazy danych) powoduje, że klasa przestaje działać
- b) Z powodu punktu a. napisano sporo testów jednostkowych, ale są one bardzo kruche, bardzo często drobna zmiana (kolejność kolumn w bazie danych, drobna zmiana w treści maila), powoduje, że całe

zestawy testów przestają działać. Sporo z testów operuje też na mockach/stubów - możesz to zobaczyć wykonując komendę: `git checkout decyzje-testy`

- c) Utrzymanie testów i klasy jest bardzo trudne - rośnie poziom jej złożoności (w bardzo szybkim tempie)
- d) Wprowadzanie nowych reguł biznesowych (lub ich zmiana), np. co do liczby dni urlopu, nowych statusów pracowników jest bardzo długa, żmudna i błędotwórcza. Dlatego, że bezpośrednio obok jest kod rozmawiający z komponentami zewnętrznymi jak `MessageBus` czy `EscalationManager`

Zaplanuj i dokonaj zmian w klasie `LegacyFighter.LeaveService`. Plan i wykonanie zmian powinno skutkować usunięciem problemów (lub największych problemów).

Przykładowe (poglądowe, nie wzorcowe) rozwiązanie znajdziesz wykonując komendę:

```
git checkout decyzje-rozwiazanie
```

Uwaga: rozwiązanie to tylko pierwszy krok. Zobacz, co teraz można zrobić z testami, które znajdziesz w klasie `LeaveServiceTest`. Czy scenariusz `slackers_do_no_get_any_leave` nie może być napisany dużo łatwiej, bez użycia obiektu `LeaveService`?